Implementation and Evaluation of QP-Based Multi-Priority Cartesian Impedance Control

1st Kadri Bajrami
Technical University of Munich
Munich, Germany
kadri.bajrami.business@gmail.com

2nd Moritz Geißler
Technical University of Munich
Munich, Germany
moritz.geissler@tum.de

I. Technical Background

This work considers torque-based multi-priority hierarchical Cartesian control of a redundant manipulator with joint configuration $\mathbf{q} \in \mathbb{R}^n$. A Cartesian task $\mathbf{x} \in \mathbb{R}^m$ is defined by

$$\mathbf{x} = \mathbf{x}(\mathbf{q}), \tag{1}$$

with differential kinematics

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}, \tag{2}$$

where $\mathbf{J} \in \mathbb{R}^{m\times n}$ denotes the task Jacobian and $m < n$ for a redundant manipulator.
The joint-space dynamics are

$$\mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \boldsymbol{\tau} + \mathbf{J}^T\mathbf{f}_{\text{ext}}, \tag{3}$$

where $\mathbf{B}$ is the inertia matrix, $\mathbf{h}$ contains Coriolis, centrifugal, and gravity terms, and $\boldsymbol{\tau}$ denotes the joint torques.
For each Cartesian objective, the desired task-space force is generated by

$$\mathbf{f}_i = \mathbf{K}_i(\mathbf{x}_i^d - \mathbf{x}_i) + \mathbf{D}_i(\dot{\mathbf{x}}_i^d - \dot{\mathbf{x}}_i), \tag{4}$$

where $\mathbf{K}_i$ and $\mathbf{D}_i$ denote task stiffness and damping. This is the simplified Cartesian impedance formulation, also referred to as Virtual Model Control in [1].

A. Hierarchical Null-Space Controller

As a traditional reference, a dynamically consistent hierarchical null-space controller is considered. For a single task, the desired force is mapped to joint torques by

$$\boldsymbol{\tau} = \mathbf{J}^T\mathbf{f}. \tag{5}$$

For redundant manipulators, additional torques can be applied in the null space of the higher-priority task. The dynamically consistent generalized inverse is

$$\bar{\mathbf{J}} = \mathbf{B}^{-1}\mathbf{J}^T\left(\mathbf{J}\mathbf{B}^{-1}\mathbf{J}^T\right)^{\dagger}, \tag{6}$$

leading to

$$\boldsymbol{\tau} = \mathbf{J}^T\mathbf{f} + \left(\mathbf{I} - \mathbf{J}^T\bar{\mathbf{J}}^T\right)\boldsymbol{\tau}_0, \tag{7}$$

where $\boldsymbol{\tau}_0$ can realize lower-priority objectives without affecting the higher-priority task.
For multiple tasks, the controller is applied recursively. With $\mathbf{N}_0 = \mathbf{I}$, the projected Jacobian is

$$\mathbf{J}_i^N = \mathbf{J}_i\mathbf{N}_{i-1}, \tag{8}$$

with corresponding inverse

$$\bar{\mathbf{J}}_i = \mathbf{B}^{-1}(\mathbf{J}_i^N)^T\left(\mathbf{J}_i^N\mathbf{B}^{-1}(\mathbf{J}_i^N)^T\right)^{\dagger}. \tag{9}$$

The null-space projector is updated according to

$$\mathbf{N}_i = \mathbf{N}_{i-1}\left(\mathbf{I} - \bar{\mathbf{J}}_i\mathbf{J}_i^N\right). \tag{10}$$

Nonlinear dynamics compensation is added to the accumulated task torque. Actuator limits are subsequently applied by element-wise saturation,

$$\boldsymbol{\tau}_d = \text{sat}\left(\boldsymbol{\tau}_{\text{motion}} + \mathbf{h}, \boldsymbol{\tau}_{\min}, \boldsymbol{\tau}_{\max}\right), \tag{11}$$

so the limits are not considered during computation of the hierarchical solution and may modify it afterwards.

B. Hierarchical QP Controller

The QP formulation proposed in [1] directly computes joint torques while preserving strict task priorities and explicitly considering constraints. For priority level i, the torque is obtained from

$$\begin{aligned}\boldsymbol{\tau}_i^* = \arg\min_{\boldsymbol{\tau}_i} \quad & \left\|\mathbf{J}_i\mathbf{B}^{-1}\boldsymbol{\tau}_i - \mathbf{J}_i\mathbf{B}^{-1}\mathbf{J}_i^T\mathbf{f}_i\right\|^2 + \epsilon\left\|\boldsymbol{\tau}_i\right\|^2 \\ \text{s.t.} \quad & \mathbf{J}_j\mathbf{B}^{-1}\boldsymbol{\tau}_i = \mathbf{J}_j\mathbf{B}^{-1}\boldsymbol{\tau}_j^*, \qquad j < i.\end{aligned} \tag{12}$$

The equality constraints preserve higher-priority solutions, while $\epsilon\|\boldsymbol{\tau}_i\|^2$ regularizes the optimized torque.
Actuator limits can be incorporated directly by

$$\boldsymbol{\tau}_{\min} - \mathbf{h} \le \boldsymbol{\tau}_i \le \boldsymbol{\tau}_{\max} - \mathbf{h}, \tag{13}$$

and the final command is

$$\boldsymbol{\tau}_d = \boldsymbol{\tau}_i^* + \mathbf{h}. \tag{14}$$

Thus, the commanded torque satisfies the actuator limits by construction, in contrast to post-hoc saturation.

II. Implementation Details

The controllers were implemented in Python and evaluated in the Genesis physics simulator using the built-in 7-DOF Franka Emika Panda. Robot kinematics and dynamics, including $\mathbf{J}$, $\mathbf{B}$, and $\mathbf{h}$, were computed using Pinocchio, while the hierarchical QPs were formulated and solved using CVXPY.
Cartesian tasks and controllers are implemented independently. Each task returns its Jacobian $\mathbf{J}_i$ and desired force $\mathbf{f}_i$ according

to (4); the x- and y-tasks use constant references, while the z-task follows sinusoidal position and velocity references.
The task list defines the priority order. One QP is solved per priority level according to (12), with previous optimal solutions added as equality constraints to subsequent levels.
Torque limits are imposed according to (13), and the final QP solution is combined with nonlinear dynamics compensation as in (14). The traditional controller also processes tasks iteratively but applies saturation only after the hierarchical solution has been computed.

III. Experimental Evaluation

Three experiments were conducted using the 7-DOF Panda with $\Delta t = 0.01$ s and a duration of 20 s. Following the task structure in [1], constant Cartesian x- and y-position objectives and a sinusoidal z-trajectory are considered.
Tracking is evaluated using Cartesian RMSE and maximum error, while RMS commanded joint torque represents control effort. Metrics exclude the first 5 s transient, whereas all plots show the full simulation.

A. Experiment 1: Non-Conflicting Tasks and Comparison with Traditional Control

The first experiment considers non-conflicting objectives without artificial torque limits,

$$x_d = 0.4 \text{ m}, \qquad y_d = 0.4 \text{ m}, \quad (15)$$

$$z_d(t) = 0.4 + 0.15 \sin(2\pi \cdot 0.1t) \text{ m}. \quad (16)$$

All tasks use $K = 20$ and $D = 10$, with gravity compensation enabled and priority order x, y, and z.
As shown in Fig. 1, both controllers successfully perform the three objectives, with faster initial convergence for the traditional controller.
The traditional controller achieves RMSE values of 7.2, 10.0, and 21.4 mm for the x, y, and z tasks, compared with 10.8, 14.2, and 30.3 mm for the QP. In contrast, the QP reduces RMS commanded torque from 17.48 to 9.25 Nm, corresponding to approximately 47%.
Average controller evaluation time is 7.86 ms for the QP and 0.085 ms for the traditional controller, demonstrating the higher computational cost of the optimization-based method.

B. Experiment 2: Conflicting Tasks and Priority Enforcement

The second experiment evaluates strict prioritization using conflicting x-position objectives $x_{d,1} = 0.4$ m and $x_{d,2} = -0.4$ m. Both use $K = 20$ and $D = 10$, with the first task assigned higher priority and no artificial torque limits.
Figure 2 shows that the higher-priority error converges approximately to zero, whereas the lower-priority task retains an error of approximately 0.8 m.
The QP obtains RMSE values of approximately 2.0 mm and 0.801 m for the high- and low-priority tasks, while the traditional controller obtains 2.2 mm and 0.799 m. Thus, both approaches successfully enforce the hierarchy.

C. Experiment 3: Performance Under Torque Constraints

The third experiment investigates active torque constraints using $K = 100$, $D = 50$, and

$$x_d = 0.4 \text{ m}, \qquad y_d = 0.4 \text{ m}, \quad (17)$$

$$z_d(t) = 0.4 + 0.5 \sin(2\pi \cdot 0.1t) \text{ m}. \quad (18)$$

All joints are limited to ± 8 Nm and gravity is disabled to isolate the influence of the torque constraints.
As shown in Fig. 3, the traditional controller saturates an initially unconstrained solution, whereas the QP optimizes directly within the available torque range.
For the sinusoidal z task, the QP reduces RMSE from 75.3 to 52.2 mm and maximum error from 188.8 to 124.1 mm, corresponding to improvements of approximately 31% and 34%. The x and y errors remain smaller for the traditional controller, indicating redistribution rather than a universal tracking improvement.
The QP also reduces RMS commanded torque from 1.68 to 1.34 Nm. Tighter limits eventually produced infeasible QPs when higher-priority constraints could no longer be satisfied.

D. Discussion

Both controllers enforce task priorities successfully, while the main QP advantage appears under actuator constraints: bounds are considered during optimization rather than through post-hoc saturation, and the objective also permits torque regularization.
This flexibility comes at higher computational cost in the present Python/CVXPY implementation. An optimized C++/qpOASES implementation in [1] operates at 1 kHz, indicating substantial potential for implementation-level improvements. Less smooth torques near active constraints could further be addressed through torque-rate penalties, warm-starting, or alternative QP solvers.

References

[1] E. M. Hoffman, A. Laurenzi, L. Muratore, N. G. Tsagarakis, and D. G. Caldwell, "Multi-Priority Cartesian Impedance Control Based on Quadratic Programming Optimization," in 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, 2018, pp. 309–315, doi: 10.1109/ICRA.2018.8462877.

[2] K. Bajrami, M. Geißler, "ARCL Project," GitHub repository, 2026. [Online]. Available: https://github.com/moerli4/arcl_project

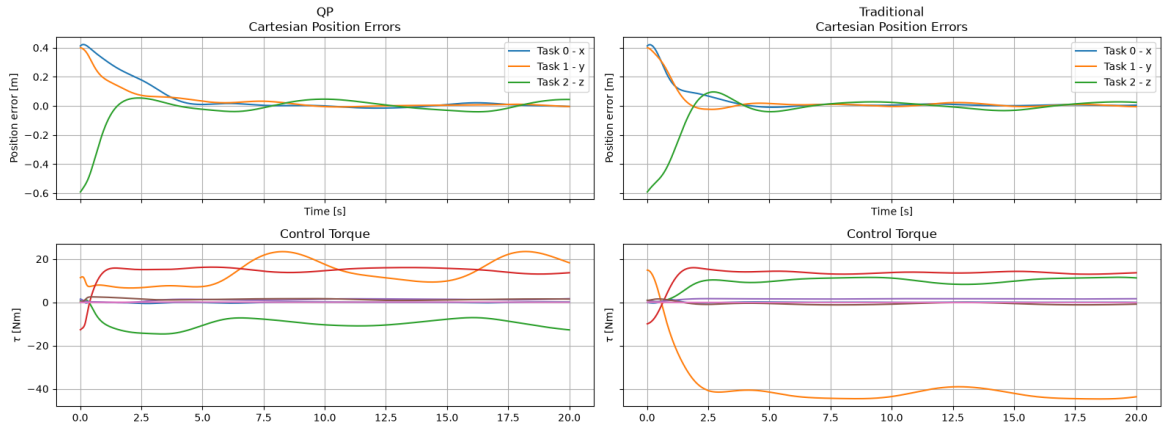


Fig. 1. Experiment 1: Cartesian errors and commanded torques for the QP controller (left) and traditional null-space controller (right).

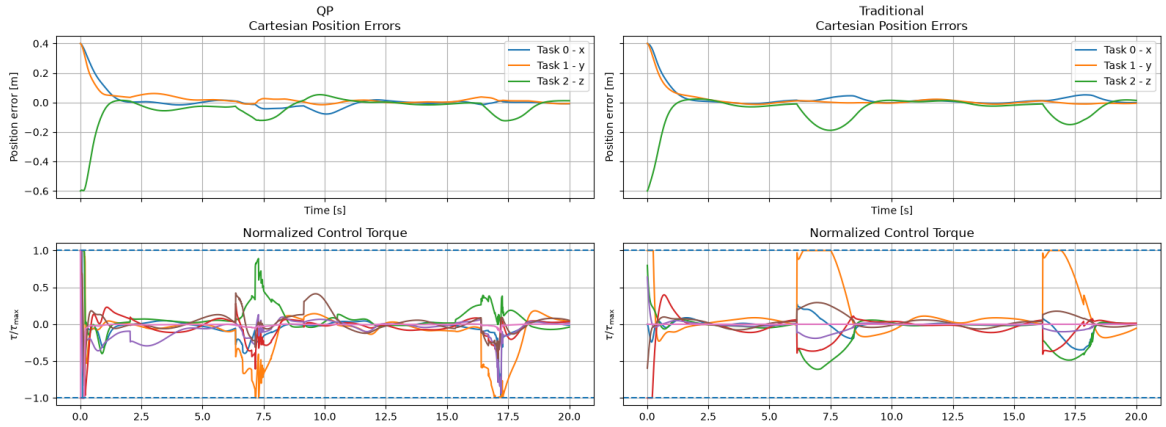


Fig. 2. Experiment 2: Response to two conflicting x-position objectives with Task 0 assigned higher priority.

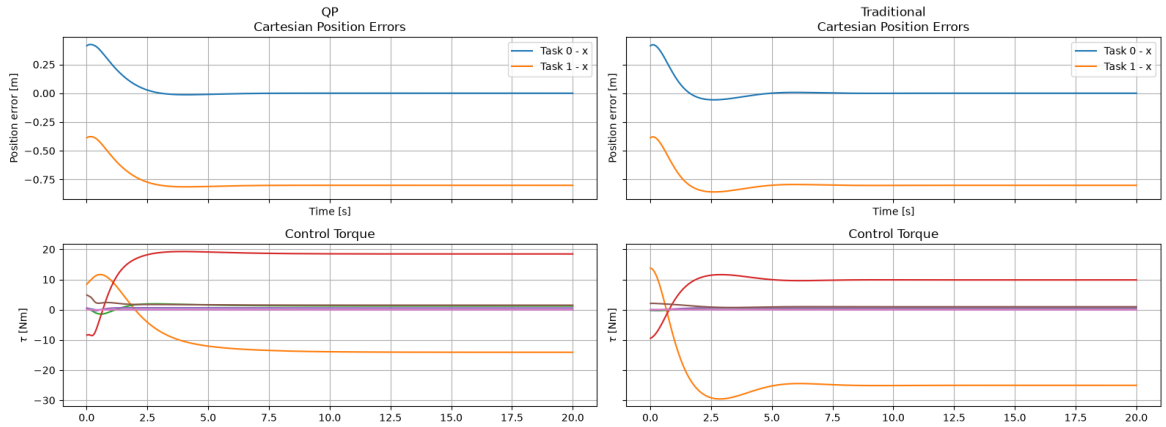


Fig. 3. Experiment 3: Cartesian errors and joint torques under artificial torque limits of ± 8 Nm.